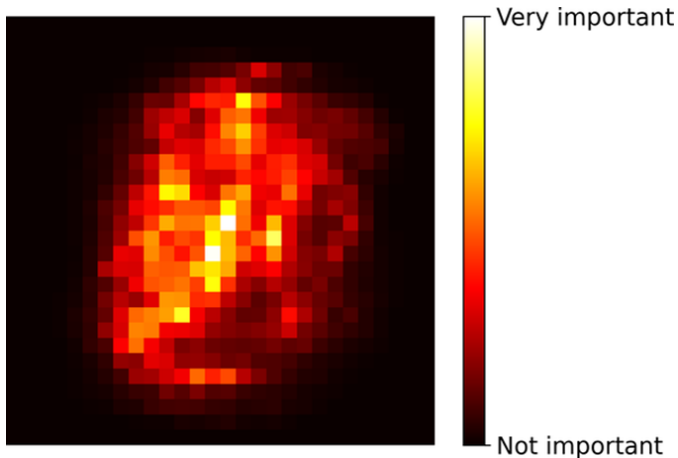


Mã nguồn: Tính toán Feature Importances

```
1 from sklearn.datasets import load_iris
2 iris = load_iris(as_frame=True)
3 rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
4 rnd_clf.fit(iris.data, iris.target)
5
6 for score, name in zip(rnd_clf.feature_importances_, iris.data.columns):
7     print(round(score, 2), name)
8
9 # Output:
10 # 0.11 sepal length (cm)
11 # 0.02 sepal width (cm)
12 # 0.44 petal length (cm)    <-- Quan trọng nhất
13 # 0.42 petal width (cm)    <-- Quan trọng thứ hai
14
```

Tầm quan trọng pixel MNIST



Hình 7-6. Tầm quan trọng của pixel trên ảnh MNIST

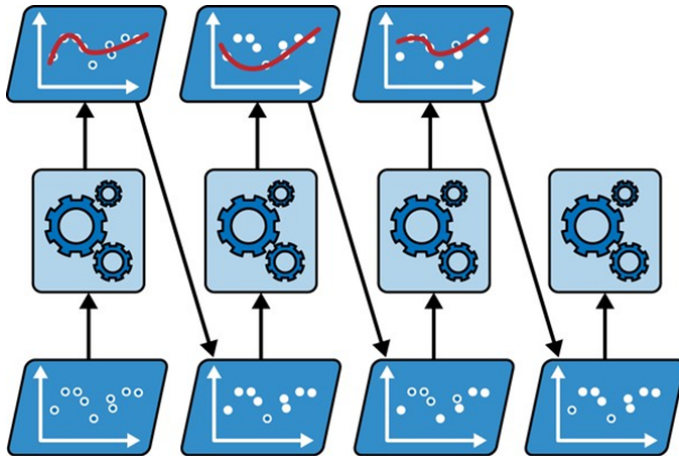
Tăng cường (Boosting)

- **Boosting** (Tăng cường giả thuyết) là phương pháp tập hợp giúp biến các bộ học yếu thành bộ học mạnh.
- Ý tưởng cốt lõi: Huấn luyện các bộ dự đoán **theo trình tự**, mỗi bộ dự đoán sau sẽ cố gắng **sửa chữa lỗi** của bộ tiền nhiệm.
- Hai phương pháp phổ biến nhất:
 - 1 AdaBoost (Adaptive Boosting)
 - 2 Gradient Boosting

AdaBoost (Adaptive Boosting)

- Cách bộ dự đoán mới sửa lỗi: **Tập trung nhiều hơn vào các trường hợp huấn luyện bị phân loại sai** bởi bộ tiền nhiệm.
- Quy trình:
 - 1 Huấn luyện mô hình cơ sở (thường là Cây quyết định một nút - decision stump) và dự đoán.
 - 2 **Tăng trọng số** tương đối của các trường hợp bị sai.
 - 3 Huấn luyện mô hình thứ 2 dựa trên trọng số mới cập nhật, rồi lại dự đoán và tăng trọng số, v.v.
- Kỹ thuật này khá giống Gradient Descent, nhưng thay vì chỉnh tham số cho một mô hình, nó thêm mô hình mới vào tập hợp.

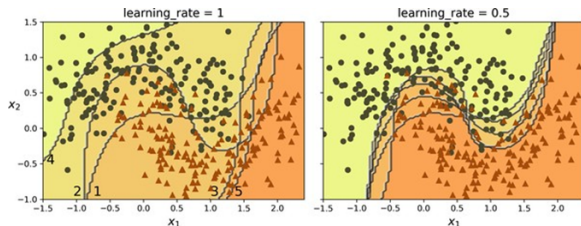
Cập nhật trọng số trong AdaBoost



Hình 7-7. Huấn luyện AdaBoost tuần tự với cập nhật trọng số

Phân tích trọng số trong SVM

- Hình bên minh họa 5 bộ SVM (RBF kernel) liên tiếp trên dữ liệu moons.
- Biểu đồ trái: Learning rate lớn, bộ thứ 2 sửa rất mạnh các lỗi của bộ đầu.
- Biểu đồ phải: Learning rate bằng một nửa, các SVM sửa đổi một cách cẩn thận và dần dần hơn.



Toán học AdaBoost: Tỷ lệ lỗi và Trọng số bộ dự đoán

- Trọng số ban đầu của mỗi trường hợp $w^{(i)} = 1/m$.
- **Tỷ lệ lỗi có trọng số của bộ thứ j (r_j):**

$$r_j = \frac{\sum_{i=1, \hat{y}_j^{(i)} \neq y^{(i)}}^m w^{(i)}}{\sum_{i=1}^m w^{(i)}} \quad (1)$$

- **Trọng số của bộ dự đoán thứ j (α_j):**

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j} \quad (2)$$

Trong đó η là learning rate (mặc định 1). Bộ càng chính xác thì α_j càng cao. Nếu đoán bừa thì $\alpha_j \approx 0$.

- Quy tắc cập nhật trọng số trường hợp ($w^{(i)}$):

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{nếu } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp(\alpha_j) & \text{nếu } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases} \quad (3)$$

- Sau đó, tất cả trọng số được chuẩn hóa (chia cho $\sum w^{(i)}$).
- Dự đoán cuối cùng:

$$\hat{y}(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \sum_{j=1, \hat{y}_j(\mathbf{x})=k}^N \alpha_j \quad (4)$$

Mã nguồn: AdaBoostClassifier

Scikit-Learn sử dụng biến thể đa lớp tên là SAMME (hoặc SAMME.R để tận dụng xác suất).

```
1 from sklearn.ensemble import AdaBoostClassifier
2 from sklearn.tree import DecisionTreeClassifier
3
4 ada_clf = AdaBoostClassifier(
5     DecisionTreeClassifier(max_depth=1), # Cây 1 nút (decision stump)
6     n_estimators=30,
7     learning_rate=0.5,
8     random_state=42
9 )
10 ada_clf.fit(X_train, y_train)
11
```

Gradient Boosting

- Tương tự AdaBoost: huấn luyện bộ sau sửa lỗi của bộ trước.
- Khác biệt: Thay vì thay đổi trọng số trường hợp, Gradient Boosting **khớp bộ dự đoán mới với các lỗi còn lại (residual errors)** do bộ dự đoán trước tạo ra.
- Gradient Tree Boosting (GBRT): Sử dụng Cây quyết định làm bộ cơ sở.

Mã nguồn: Gradient Tree Boosting thủ công (Phần 1)

Tạo dữ liệu và huấn luyện cây đầu tiên:

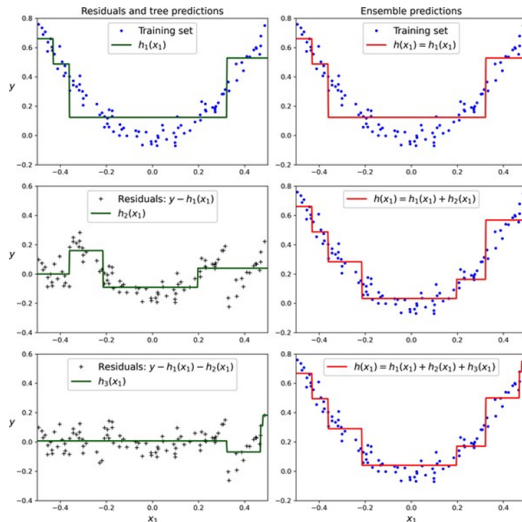
```
1 from sklearn.tree import DecisionTreeRegressor
2 import numpy as np
3
4 np.random.seed(42)
5 X = np.random.rand(100, 1) - 0.5
6 y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100)
7
8 tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
9 tree_reg1.fit(X, y)
10
```

Mã nguồn: Gradient Tree Boosting thủ công (Phần 2)

Huấn luyện trên phần dư (residuals) và kết hợp:

```
1 y2 = y - tree_reg1.predict(X) # Phan du 1
2 tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
3 tree_reg2.fit(X, y2)
4
5 y3 = y2 - tree_reg2.predict(X) # Phan du 2
6 tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
7 tree_reg3.fit(X, y3)
8
9 # Du doan moi bang tong cac cay:
10 X_new = np.array([[-0.4], [0.], [0.5]])
11 y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
12
```

Quá trình huấn luyện GBRT



Hình 7-9. Các dự đoán của từng cây (trái) và tổng hợp của tập hợp (phải)

GradientBoostingRegressor trong Scikit-Learn

Sử dụng lớp có sẵn để tự động quá trình này:

```
1 from sklearn.ensemble import GradientBoostingRegressor
2
3 gbrt = GradientBoostingRegressor(
4     max_depth=2,
5     n_estimators=3,
6     learning_rate=1.0,
7     random_state=42
8 )
9 gbrt.fit(X, y)
10
```

Vấn đề quá khớp và Shrinkage

- `learning_rate`: Kiểm soát mức độ đóng góp của từng cây.
- Nếu đặt giá trị thấp (ví dụ 0.05), sẽ cần nhiều cây hơn để khớp dữ liệu (Shrinkage - một phương pháp chính quy hóa).
- Nếu số lượng cây `n_estimators` quá ít → Dưới khớp.
- Nếu số lượng cây `n_estimators` quá nhiều → Quá khớp.